

Module 12 - chapter 5

Change & Destroy Terraform Resources

Changing a resource

In the resources created in Module 12 -chapter 4 we didn't give them names:

New subnet in default VPC has no name:

Subnets (1/4) [Info](#)

VPC = vpc-03677c88f7cc2f715

☐	Name	Subnet ID
☐	-	subnet-03da2ce00b3dba523
☒	-	subnet-0b95d93084673c81b
☐	-	subnet-092c85ea013cbc240
☐	-	subnet-051c96240932f9c7e

New VPC has no name:

Your VPCs	
VPCs	VPC encryption controls
Your VPCs (1/2) Info	
Find VPCs by attribute or tag	
☐ Name	VPC ID
☒ -	vpc-0e43e232c4a94ae2a
☐ -	vpc-03677c88f7cc2f715

So we will make that change in our configuration. To add names we use a parameter called 'tags' which are key-value pair in AWS

```

7 resource "aws_vpc" "development-vpc" {
8   cidr_block = "10.0.0.0/16"
9   tags = {
10
11   }
12 }
13
14 resource "aws_subnet" "dev-subnet-1" {
15   vpc_id = aws_vpc.development-vpc.id

```

tags = key-value pairs in AWS

And we can give any word we want to the key and the value -> myKey : myValue but for name there is a key attribute reserved which is 'Name'

This is the VPC with the tags -> name attribute change

```

6
7 resource "aws_vpc" "development-vpc" {
8   cidr_block = "10.0.0.0/16"
9   tags = {
10     Name: "development"
11   }
12 }

```

You, Moments ago • Uncommi

Now we do the same for the subnets:

```

15 resource "aws_subnet" "dev-subnet-1" {
16     vpc_id = aws_vpc.development-vpc.id
17     cidr_block = "10.0.10.0/24"
18     availability_zone = "eu-west-2a"
19     tags = {
20         Name: "subnet-1-dev"
21     }
22 }
23 | Pardo, Yesterday • add AWS resources and data
24 data "aws_vpc" "existing_vpc" {
25     default = true
26 }
27
28 resource "aws_subnet" "dev-subnet-2" {
29     vpc_id = data.aws_vpc.existing_vpc.id
30     cidr_block = "172.31.48.0/20"
31     availability_zone = "eu-west-2a"
32     tags = {
33         Name: "subnet-2-default"
34     }
35 }

```

And we add another tag to the VPC and we decided to call it 'vpc_env'

```

7 resource "aws_vpc" "development-vpc" {
8     cidr_block = "10.0.0.0/16"
9     tags = {
10         Name: "development",
11         vpc_env: "dev"
12     }
13 }

```

-> terraform apply

And we get

```
→ terraform git:(Module_12/Terraform_chapter-5) × terraform apply
data.aws_vpc.existing_vpc: Reading...
aws_vpc.development-vpc: Refreshing state... [id=vpc-0e43e232c4a94ae2a]
data.aws_vpc.existing_vpc: Read complete after 1s [id=vpc-03677c88f7cc2f715]
aws_subnet.dev-subnet-2: Refreshing state... [id=subnet-0b95d93084673c81b]
aws_subnet.dev-subnet-1: Refreshing state... [id=subnet-0ed81ef156fc90fa9]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following
symbols:
~ update in-place

Terraform will perform the following actions:

# aws_subnet.dev-subnet-1 will be updated in-place
~ resource "aws_subnet" "dev-subnet-1" {
  id              = "subnet-0ed81ef156fc90fa9"
  ~ tags          = {
    + "Name" = "subnet-1-dev"
  }
  ~ tags_all      = {
    + "Name" = "subnet-1-dev"
  }
  # (20 unchanged attributes hidden)
}
```

We can see that now we have the '~' symbol meaning change -> update existing resources

So we can see that 'dev-subnet-1' will be changed, we will change the 'tags' attribute by adding '+' a 'Name'

We can also see that there are 20 hidden attributes that will remain unchanged

```
~ resource "aws_subnet" "dev-subnet-1" {
  id              = "subnet-0ed81ef156fc90fa9"
  ~ tags          = {
    + "Name" = "subnet-1-dev"
  }
  ~ tags_all      = {
    + "Name" = "subnet-1-dev"
  }
  # (20 unchanged attributes hidden)
}
```

And we get similar information about the changes for 'dev-subnet-2' and 'development-vpc'

```

# aws_subnet.dev-subnet-2 will be updated in-place
~ resource "aws_subnet" "dev-subnet-2" {
  id = "subnet-0b95d93084673c81b"
  ~ tags = {
    + "Name" = "subnet-2-default"
  }
  ~ tags_all = {
    + "Name" = "subnet-2-default"
  }
  # (20 unchanged attributes hidden)
}

# aws_vpc.development-vpc will be updated in-place
~ resource "aws_vpc" "development-vpc" {
  id = "vpc-0e43e232c4a94ae2a"
  ~ tags = {
    + "Name" = "development"
    + "vpc_env" = "dev"
  }
  ~ tags_all = {
    + "Name" = "development"
    + "vpc_env" = "dev"
  }
  # (19 unchanged attributes hidden)
}

```

And we get the summary of the plan:

Plan: 0 to add, 3 to change, 0 to destroy.

Do you want to perform these actions?

Terraform will perform the actions described above.

Only 'yes' will be accepted to approve.

Enter a value:

So we type -> yes

```
Plan: 0 to add, 3 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

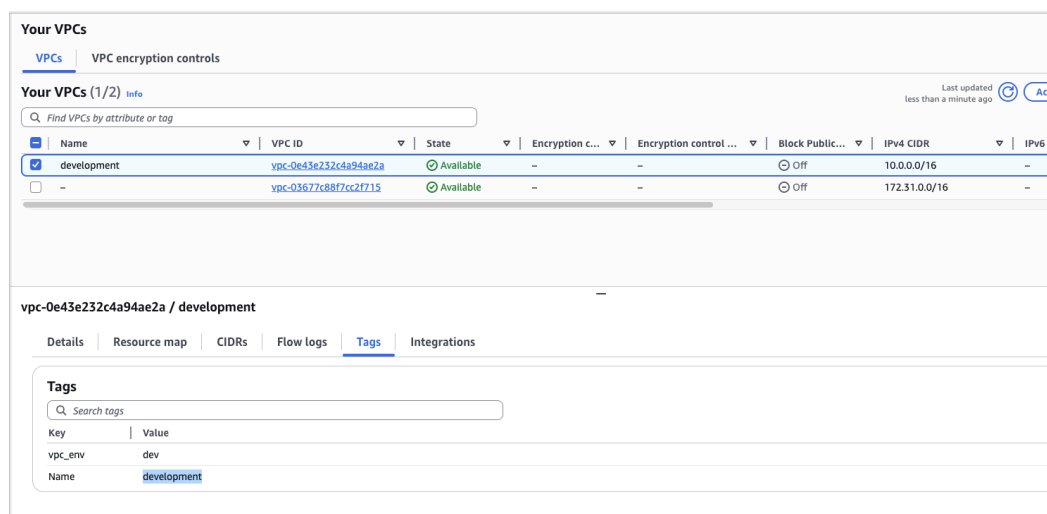
  Enter a value: yes

aws_vpc.development-vpc: Modifying... [id=vpc-0e43e232c4a94ae2a]
aws_subnet.dev-subnet-2: Modifying... [id=subnet-0b95d93084673c81b]
aws_subnet.dev-subnet-2: Modifications complete after 0s [id=subnet-0b95d93084673c81b]
aws_vpc.development-vpc: Modifications complete after 1s [id=vpc-0e43e232c4a94ae2a]
aws_subnet.dev-subnet-1: Modifying... [id=subnet-0ed81ef156fc90fa9]
aws_subnet.dev-subnet-1: Modifications complete after 0s [id=subnet-0ed81ef156fc90fa9]

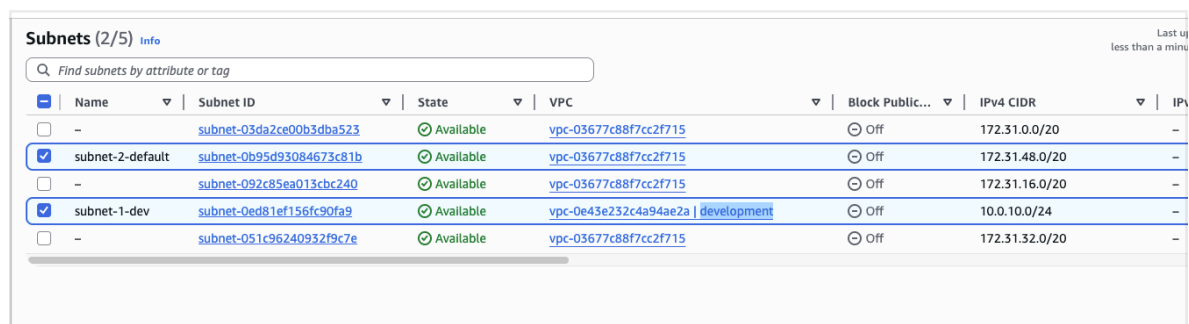
Apply complete! Resources: 0 added, 3 changed, 0 destroyed.
→ terraform git:(Module_12/Terraform_chapter-5) ×
```

We can see that 3 changes have been made.

And if we check AWS UI we see the VPC has now a name 'development' and also two tags : Name and vpc_env



And the subnets got updated as well



Remove something from a resource

Let's say we want to remove the tag `vpc_env` from VPC 'development-vpc'

```
7 resource "aws_vpc" "development-vpc" {
8     cidr_block = "10.0.0.0/16"
9     tags = {
10         Name: "development"
11     }
12 }
```

-> terraform apply

```
+ terraform git:(Module_12/Terraform_chapter-5) x terraform apply
data.aws_vpc.existing_vpc: Reading...
aws_vpc.development-vpc: Refreshing state... [id=vpc-0e43e232c4a94ae2a]
data.aws_vpc.existing_vpc: Read complete after 1s [id=vpc-03677c88f7cc2f715]
aws_subnet.dev-subnet-2: Refreshing state... [id=subnet-0b95d93084673c81b]
aws_subnet.dev-subnet-1: Refreshing state... [id=subnet-0ed81ef156fc98fa9]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following
symbols:
  ~ update in-place

Terraform will perform the following actions:

# aws_vpc.development-vpc will be updated in-place
~ resource "aws_vpc" "development-vpc" {
  id           = "vpc-0e43e232c4a94ae2a"
  ~ tags       = {
    "Name"     = "development"
    - "vpc_env" = "dev" -> null
  }
  ~ tags_all   = {
    - "vpc_env" = "dev" -> null
    # (1 unchanged element hidden)
  }
  # (19 unchanged attributes hidden)
}
```

Now we can see there is a change '~' and the change will remove '-' 'vpc_env' tag from the resource

```
Plan: 0 to add, 1 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

Enter a value: yes

aws_vpc.development-vpc: Modifying... [id=vpc-0e43e232c4a94ae2a]
aws_vpc.development-vpc: Modifications complete after 0s [id=vpc-0e43e232c4a94ae2a]

Apply complete! Resources: 0 added, 1 changed, 0 destroyed.
```

And we check AWS UI

Your VPCs

VPCs VPC encryption controls

Your VPCs (1/2) Info Last updated less than a minute ago

Find VPCs by attribute or tag

Name	VPC ID	State	Encryption c...	Encryption control ...	Block Public...	IPv4 CIDR
development	vpc-0e43e232c4a94ae2a	Available	-	-	Off	10.0.0.0/16
-	vpc-03677c88f7cc2f715	Available	-	-	Off	172.31.0.0/16

vpc-0e43e232c4a94ae2a / development

Details Resource map CIDRs Flow logs **Tags** Integrations

Tags

Search tags

Key	Value
Name	development

And the tag has been removed.

Removing - Destroying resources

Let's say we want to remove the whole resource for example 'dev-subnet-2'.

There are two ways

1. Remove the code from tf file and -> terraform apply

```

12 }
13
14 resource "aws_subnet" "dev-subnet-1" {
15     vpc_id = aws_vpc.development-vpc.id
16     cidr_block = "10.0.10.0/24"
17     availability_zone = "eu-west-2a"
18     tags = {
19         Name: "subnet-1-dev"
20     }
21 }
22
23 #data "aws_vpc" "existing_vpc" { You, 24 minutes ago • Uncommitted changes
24     # default = true
25     #}
26     #
27     #resource "aws_subnet" "dev-subnet-2" {
28     #     vpc_id = data.aws_vpc.existing_vpc.id
29     #     cidr_block = "172.31.48.0/20"
30     #     availability_zone = "eu-west-2a"
31     #     tags = {
32     #         Name: "subnet-2-default"
33     #     }
34     #}

```

2. Using command line

-> terraform destroy -target <resourceType>.<resourceName>

-> terraform destroy -target aws_subnet.dev-subnet-2

```
+ terraform git:(Module_12/Terraform_chapter-5) × terraform destroy -target aws_subnet.dev-subnet-2
data.aws_vpc.existing_vpc: Reading...
data.aws_vpc.existing_vpc: Read complete after 1s [id=vpc-03677c88f7cc2f715]
aws_subnet.dev-subnet-2: Refreshing state... [id=subnet-0b95d93084673c81b]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following
symbols:
  - destroy

Terraform will perform the following actions:

# aws_subnet.dev-subnet-2 will be destroyed
resource "aws_subnet" "dev-subnet-2" {
  - arn                                     = "arn:aws:ec2:eu-west-2:706976333940:subnet/subnet-0b95d93084673c81b" -> null
  - assign_ipv6_address_on_creation        = false -> null
  - availability_zone                     = "eu-west-2a" -> null
  - availability_zone_id                  = "euw2-az2" -> null
  - cidr_block                            = "172.31.48.0/20" -> null
  - enable_dns64                          = false -> null
  - enable_lni_at_device_index            = 0 -> null
  - enable_resource_name_dns_a_record_on_launch = false -> null
  - enable_resource_name_dns_aaaa_record_on_launch = false -> null
  - id                                    = "subnet-0b95d93084673c81b" -> null
  - ipv6_native                           = false -> null
  - map_customer_owned_ip_on_launch       = false -> null
```

And we can see that a whole resource will be destroyed '-'

```
  - map_customer_owned_ip_on_launch       = false -> null
  - map_public_ip_on_launch               = false -> null
  - owner_id                             = "706976333940" -> null
  - private_dns_hostname_type_on_launch   = "ip-name" -> null
  - region                                = "eu-west-2" -> null
  - tags                                  = {
    - "Name" = "subnet-2-default"
  } -> null
  - tags_all                              = {
    - "Name" = "subnet-2-default"
  } -> null
  - vpc_id                                = "vpc-03677c88f7cc2f715" -> null
  # (4 unchanged attributes hidden)
}

Plan: 0 to add, 0 to change, 1 to destroy.

Warning: Resource targeting is in effect

You are creating a plan with the -target option, which means that the result of this plan may not represent all of the changes
requested by the current configuration.

The -target option is not for routine use, and is provided only for exceptional situations such as recovering from errors or mistakes,
or when Terraform specifically suggests to use it as part of an error message.
```

And I confirm -> yes

```
Do you really want to destroy all resources?
Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.

Enter a value: yes

aws_subnet.dev-subnet-2: Destroying... [id=subnet-0b95d93084673c81b]
aws_subnet.dev-subnet-2: Destruction complete after 1s

Warning: Applied changes may be incomplete

The plan was created with the -target option in effect, so some changes requested in the configuration may have been ignored and the
output values may not be fully updated. Run the following command to verify that no other changes are pending:
    terraform plan

Note that the -target option is not suitable for routine use, and is provided only for exceptional situations such as recovering from
errors or mistakes, or when Terraform specifically suggests to use it as part of an error message.

Destroy complete! Resources: 1 destroyed.
+ terraform git:(Module_12/Terraform_chapter-5) ×
```

The resource has been destroyed and we can check AWS UI

	Name	Subnet ID	State	VPC	Block Public...	IPv4 CIDR
☐	-	subnet-03da2ce0b3dba523	Available	vpc-03677c88f7cc2f715	Off	172.31.0.0/20
☐	-	subnet-092c85ea013cbc240	Available	vpc-03677c88f7cc2f715	Off	172.31.16.0/20
☒	subnet-1-dev	subnet-0ed81ef156fc90fa9	Available	vpc-0e43e232c4a94ae2a development	Off	10.0.10.0/24
☐	-	subnet-051c96240932f9c7e	Available	vpc-03677c88f7cc2f715	Off	172.31.32.0/20

But now my tf file has code/ configuration that doesn't match the current state (the state file was updated when executing terraform destroy -target aws_subnet.dev-subnet-2). So best practice is to always use the first way -> terraform apply



now my state file is different than my tf file which still has the resource and if I run terraform apply the resource will be created again.

that's the problem with -target — it creates a mismatch between our state file and our .tf config.

The correct flow is:

1. Remove the resource from your .tf file
2. Run terraform apply

Terraform will see the resource is in the state file but no longer in the config and delete it — keeping everything in **sync**.

-target is generally only used for emergencies or very specific situations, not as a regular workflow.

Questions

Q:

I created a tag using terraform, then I deleted the tag from the tf file and execute terraform apply so now the tag is gone. my AWS account had a default vpc created before I used terraform for the first time. how terraform knows not to remove the default vpc if is not in the tf file like it did with the tag

A:

Terraform only manages resources it created or was explicitly told to manage.

The default VPC was created by AWS, not by Terraform — so Terraform has no record of it in its state file. Terraform's rule is:

■ "If it's not in my state file, it doesn't exist to me"

The tag was different — you created it with Terraform, so it was tracked in the state file. When you removed it from the .tf file and ran apply, Terraform saw the difference between the state and your config and deleted it.

To manage an existing resource (like the default VPC) with Terraform you'd have to explicitly import it:

```
terraform import aws_vpc.default <vpc-id>
```

Only then would Terraform start tracking and managing it.

Q:

ok, so if I create a new subnet to the default vpc using tf then if I remove the resource from tf then the subnet will be removed?

A:

Exactly. Because you created the subnet with Terraform, it lives in the state file. So when you remove it from your .tf file and run terraform apply, Terraform sees:

- State file says: subnet exists
- Config says: subnet not defined
- Action: delete it

Same logic as the tag.

Q:

so when I run terraform apply tf creates a state file and when I run terraform

apply again it checks the state file then it executes changes to AWS then it creates a new state file. and this is how it knows what was in tf file before the code editing

A:

Exactly right. The flow is:

1. First terraform apply → creates resources in AWS + writes state file
2. You edit .tf files
3. Second terraform apply → compares your new config against the state file, makes only the necessary changes in AWS, then updates the state file

The state file is essentially Terraform's memory of "what I built and what it looks like right now."

One thing to add — terraform plan shows you the diff between state and config before applying, which is useful to verify what will be created/changed/deleted before actually doing it.

Q:

so if I create a vpc with tf and then manually in AWS I add a new subnet to it. tf won't see it as it won't be in the state file

A:

Exactly. Terraform is unaware of it because:

- It's not in the state file (Terraform didn't create it)
- It's not in your .tf config

So if you then run terraform apply it won't touch that subnet at all — it just ignores it.

But there's a risk — if you run terraform apply on the VPC itself (say you change a tag), Terraform only manages what it knows about. The manually created subnet stays untouched.

However if you were to destroy the VPC with Terraform (terraform destroy), AWS would also delete the subnet because it belongs to that VPC — even though Terraform didn't know about it.

Q:

so the state file is not tf going around AWS looking for AWS account current state. the state file is a file made from a tf file itself

A:

Exactly. The state file is built from what Terraform itself has done, not from scanning your AWS account.

Think of it as Terraform's own notebook — it only writes down things it created. It has no knowledge of anything that exists in AWS outside of that.

If you want Terraform to scan your actual AWS account and compare it against the state file you can run:

-> terraform refresh

That updates the state file to reflect the real current state of resources Terraform already tracks — but it still won't discover resources it never created.